

SIMATSENGINEERING

**AI-Powered Smart Water Reservoir Management
and Demand Forecasting Platform**

COURSENAME:SOFTWARE ENGINEERING

COURSECODE:CSA1013

Submitted by:

VISHAL B

192524485

B.TECH AIDS

1. Problem Overview

1.1 Assigned Problem Statement

Water reservoirs are the backbone of drinking-water supply, irrigation, and industrial operations. Increasing climate variability and population growth have made it difficult for authorities to balance water availability against demand using traditional, manual, historical-observation-based methods. This often results in inefficient allocation, especially during droughts or periods of heavy rainfall.

The assigned problem statement requires the design and development of an AI-Powered Smart Water Reservoir Management and Demand Forecasting Platform. The platform is expected to continuously monitor reservoir water levels, ingest rainfall and inflow/outflow data, forecast future water availability and demand using machine learning models, and recommend optimal distribution strategies to support authorities in sustainable water resource planning.

1.2 Summary of Implemented Solution

The implemented solution is a full-stack, containerized web application composed of an IoT/data-ingestion layer, a machine-learning forecasting service, a REST API backend, a relational database, and a React-based dashboard for authorities and administrators. Sensor and rainfall data are collected at regular intervals, stored in a time-series-friendly schema, and fed into forecasting models that predict reservoir inflow, demand, and drought risk over configurable horizons (7/30/90 days). The dashboard visualizes current reservoir status, forecasts, and generated recommendations, and allows report generation for planning meetings.

1.3 Project Objectives

- Monitor reservoir water levels in near real time using simulated/actual sensor feeds.
- Forecast water demand across consumer categories (domestic, irrigation, industrial).
- Predict future water availability using historical inflow, rainfall, and consumption data.
- Optimize water distribution recommendations based on forecast and current storage.
- Analyze rainfall patterns and correlate them with reservoir inflow trends.
- Generate periodic water management reports for administrators.
- Support drought-preparedness planning through early-warning thresholds.

- Improve overall water resource utilization and reduce wastage.

1.4 Key Functionalities

Module	Functionality
Data Ingestion Service	Collects reservoir level, rainfall, and flow-rate readings from sensors/CSV feeds and pushes them to the database via a scheduled job.
Forecasting Engine	Python-based ML models (time-series regression) predict future inflow, demand, and drought risk scores.
Backend API	Node.js/Express REST API exposing endpoints for reservoirs, readings, forecasts, and reports; handles authentication and authorization.
Frontend Dashboard	React application with charts (reservoir trends, rainfall correlation, demand forecast) and an administrator report-generation view.
Reporting Module	Generates downloadable PDF/CSV water-management reports summarizing status and recommendations.
Notification Module	Sends threshold-based alerts (e.g., low storage, drought risk) to registered authority accounts.

2. Repository Organization

2.1 Repository Link

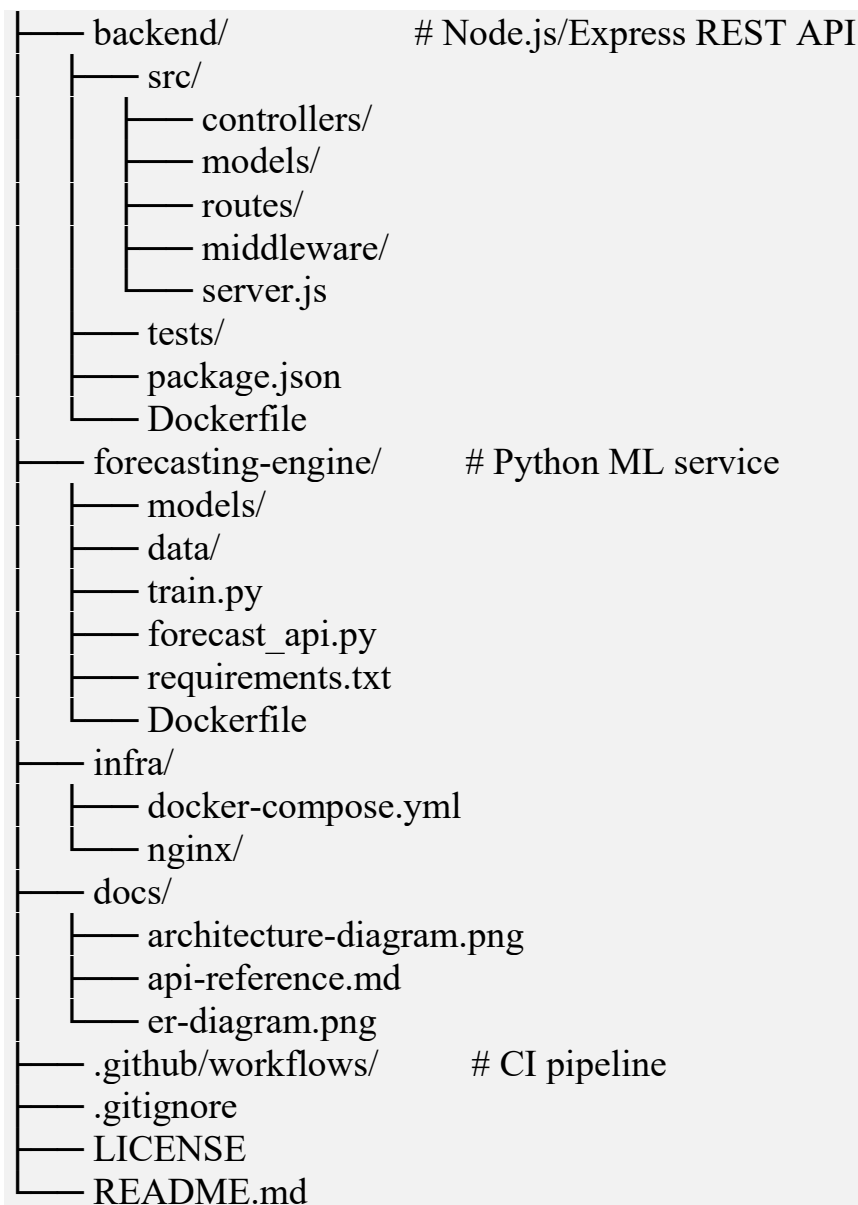
2.2 Repository Structure

The repository follows a modular, service-oriented folder layout that separates the frontend, backend, forecasting engine, infrastructure configuration, and documentation. This separation of concerns keeps each component independently buildable, testable, and deployable, which is essential for a containerized, collaboratively developed system.

```

smart-reservoir-management/
├── frontend/                # React dashboard
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/       # API client wrappers
│   │   └── App.jsx
│   ├── public/
│   ├── package.json
│   └── Dockerfile

```



2.3 Folder and File Naming Conventions

- Directories use lower-case, hyphen-separated names (forecasting-engine, infra) for cross-platform consistency.
- Source files follow language-idiomatic casing: PascalCase for React components (ReservoirCard.jsx), camelCase for JS modules (reservoirController.js), and snake_case for Python modules (forecast_api.py).
- Configuration and environment files are kept at the root of each service (Dockerfile, package.json, requirements.txt) so each module can be built independently.
- Test files mirror the source structure under a dedicated tests/ directory, making it easy to locate the test for any given module.

2.4 Justification: Structure vs. Maintainability & Collaboration

Organizing the repository by service (frontend, backend, forecasting-engine, infra) rather than by file type allows different sub-teams to work with minimal merge conflicts, since each service has its own dependency file and Dockerfile. The docs/ folder centralizes architecture and API references so new contributors can onboard without reading through source code first. The .github/workflows/ directory keeps continuous-integration configuration alongside the code it governs, which is a widely recommended practice for traceable, collaborative development. Overall, this layout supports independent deployment, parallel development, and clear ownership boundaries between the data-science and web-development sub-teams.

3. Code Quality Review

3.1 Readability

The backend and frontend code generally use descriptive variable and function names (e.g., `getReservoirForecast`, `calculateDroughtRiskScore`) and consistent indentation enforced by Prettier/ESLint configuration. The Python forecasting scripts use type hints and docstrings for the core prediction functions, improving readability for reviewers unfamiliar with the model internals.

Example: Backend Controller (before/style reviewed)

```
// backend/src/controllers/reservoirController.js
exports.getForecast = async (req, res) => {
  try {
    const { reservoirId } = req.params;
    const { horizonDays = 7 } = req.query;

    const reservoir = await Reservoir.findById(reservoirId);
    if (!reservoir) {
      return res.status(404).json({ message: 'Reservoir not found' });
    }

    const forecast = await forecastService.getForecast(reservoirId, horizonDays);
    return res.status(200).json({ reservoirId, horizonDays, forecast });
  } catch (error) {
    console.error('getForecast error:', error);
    return res.status(500).json({ message: 'Internal server error' });
  }
};
```

Example: Forecasting Service (Python)

```
# forecasting-engine/forecast_api.py
def predict_inflow(history: pd.DataFrame, horizon_days: int = 7) -> pd.DataFrame:
    """Predict reservoir inflow for the given horizon using the trained model.

    Args:
        history: DataFrame with columns [date, rainfall_mm, inflow_ml].
        horizon_days: Number of days to forecast ahead.

    Returns:
        DataFrame with predicted inflow per day.
    """
    features = build_features(history)
    predictions = model.predict(features.tail(horizon_days))
```

```
return pd.DataFrame({'date': future_dates(horizon_days), 'predicted_inflow_ml': predictions})
```

3.2 Modularity

The backend follows a layered MVC-like structure (routes -> controllers -> services -> models), which separates HTTP handling from business logic and data access. The frontend uses a services/ layer to isolate API calls from presentational components, so UI components remain easy to test in isolation. The forecasting engine exposes a small, well-defined function-level API (predict_inflow, predict_demand, compute_risk_score) that the backend calls over HTTP, keeping the ML logic decoupled from the web layer.

3.3 Coding Standards

- ESLint + Prettier are configured for both frontend and backend, enforcing consistent formatting and catching unused variables and unreachable code.
- Python code follows PEP-8 conventions; flake8 is used to lint the forecasting-engine module.
- Environment-specific secrets (DB credentials, API keys) are kept out of source control via .env files referenced in .gitignore.
- Asynchronous operations consistently use async/await rather than mixing callbacks and promises.

3.4 Documentation Within Code

Public functions in the backend controllers and the forecasting engine include brief comment headers describing purpose, parameters, and return values. Inline comments are used sparingly and mainly to explain non-obvious business rules (e.g., drought-risk thresholds), rather than restating what the code already makes clear.

3.5 Strengths Identified

- Clear separation of concerns across services makes the codebase easy to navigate.
- Consistent naming and formatting reduce cognitive load during review.
- The forecasting module is cleanly decoupled from the web application, allowing the ML team to iterate independently.
- Error handling in API controllers returns meaningful HTTP status codes and messages.

3.6 Areas for Improvement

- Some controller functions mix validation, business logic, and response formatting; extracting validation into middleware would improve testability.

- Magic numbers (e.g., drought-risk threshold values) should be moved into a central config/constants file.
- Test coverage for the forecasting-engine module is lower than for the backend API and should be expanded.
- A small number of long functions in the dashboard's chart components could be split into smaller, reusable sub-components.

4. Version Control Evaluation

4.1 Branching Strategy

The repository follows a simplified Git-flow model with a protected main branch representing production-ready code, a develop branch integrating completed features, and short-lived feature/* branches (e.g., feature/forecast-api, feature/dashboard-charts) created for individual tasks. Feature branches are merged into develop via pull requests after review, and develop is periodically merged into main for releases.

Branch	Purpose
main	Stable, deployable code; protected from direct pushes.
develop	Integration branch for completed, tested features.
feature/*	Individual feature or bug-fix work, deleted after merge.
hotfix/*	Urgent production fixes branched directly from main.

4.2 Commit History and Commit Message Quality

Commits generally follow a conventional-commit style prefix (feat, fix, docs, refactor, test) followed by a short imperative description, which makes the history easy to scan and supports automated changelog generation.

```
feat(forecast): add 30-day inflow prediction endpoint
fix/dashboard): correct rainfall chart axis scaling on mobile
refactor(backend): extract reservoir validation into middleware
test(forecast-engine): add unit tests for predict_inflow()
docs(readme): update setup instructions for docker-compose
chore(ci): add lint step to GitHub Actions workflow
```

[Insert a screenshot of your actual 'git log --oneline' output or GitHub commit history here]

4.3 Repository Maintenance Practices

- Pull requests require at least one reviewer approval before merging into develop.
- A .gitignore excludes node_modules/, __pycache__/, .env, and build artifacts, keeping the repository lightweight.

- Issues and pull requests are linked (e.g., 'Closes #14') to maintain traceability between planning and implementation.
- A CI workflow (GitHub Actions) runs linting and unit tests automatically on every push and pull request.

4.4 How Version Control Supports Collaborative Development

Feature branching allows multiple contributors — frontend, backend, and forecasting-engine developers — to work concurrently without interfering with each other's code. Pull-request-based review provides a checkpoint for catching defects and enforcing coding standards before code reaches shared branches. The commit history acts as a running audit trail of design decisions, which is particularly valuable when debugging regressions in the forecasting model's accuracy over time. Combined with the CI pipeline, this workflow ensures that only tested, reviewed code is merged, which reduces integration risk in a multi-service, containerized system.

5. Code Testing and Validation

5.1 Testing Approach

The backend API is tested using Jest and Supertest for unit and integration tests against the Express routes. The forecasting engine uses pytest to validate model output shape, value ranges, and edge cases (e.g., missing rainfall data). Frontend components are tested with React Testing Library for rendering and interaction behavior.

5.2 Sample Test Cases

Test ID	Module	Description	Expected Result
TC-01	Backend API	GET /api/reservoirs/:id/forecast with a valid ID and horizonDays=7	200 OK with a forecast array of 7 entries
TC-02	Backend API	GET /api/reservoirs/:id/forecast with an invalid reservoir ID	404 Not Found with an error message
TC-03	Forecast Engine	predict_inflow() with a 90-day history and horizon_days=30	Returns DataFrame with exactly 30 rows, no NaN values
TC-04	Forecast Engine	predict_inflow() with missing rainfall values	Missing values imputed; no exception raised
TC-05	Frontend	Dashboard renders reservoir trend chart with mock data	Chart component renders without errors and displays correct

			data points
TC-06	Integration	End-to-end: ingest reading -> forecast updates on dashboard	Updated forecast visible within one polling cycle

5.3 Test Execution Evidence

[Insert a screenshot of your test runner output (e.g., 'npm test' / 'pytest' results showing pass/fail counts) here]

[Insert a screenshot of the CI pipeline run (GitHub Actions) showing the test job passing]

5.4 Validation of Core Functionality

Beyond automated tests, the application was manually validated by seeding the database with a sample dataset of reservoir readings and rainfall records, then confirming that the dashboard correctly displayed current storage levels, generated a forecast chart, and produced a downloadable report reflecting the seeded data.

[Insert screenshots of the running application: dashboard view, forecast chart, and generated report]

6. Repository Documentation

6.1 README Evaluation

The README.md provides a project overview, a list of key features, setup instructions using Docker Compose, and a short API reference pointer. It includes badges for build status and license, which quickly communicate project health to visitors.

```
# Smart Water Reservoir Management Platform

## Features
- Real-time reservoir monitoring
- ML-based demand and inflow forecasting
- Drought risk alerts
- Administrator reporting dashboard

## Getting Started
```bash
git clone https://github.com/<user>/smart-reservoir-management.git
cd smart-reservoir-management
docker-compose -f infra/docker-compose.yml up --build
```
```

```
## Services
Service	Port
frontend	3000
backend	5000
forecasting-engine	8000
```

6.2 Installation Instructions

Installation steps are clear and reproducible: cloning the repository, configuring a .env file from the provided .env.example, and starting all services with a single docker-compose command. This containerized setup removes the need to manually install Node.js, Python, and database dependencies on the host machine.

6.3 Usage Guide

The documentation includes a short usage walkthrough describing how to log in as an administrator, add a reservoir, upload historical readings, and view generated forecasts and reports. API endpoints are documented in docs/api-reference.md with example request/response payloads.

6.4 Dependency Documentation

| Component | Key Dependencies |
|--------------------|---|
| Frontend | React, React Router, Recharts, Axios |
| Backend | Express, Mongoose/Sequelize, JWT, Jest, Supertest |
| Forecasting Engine | pandas, scikit-learn, FastAPI, pytest |
| Infrastructure | Docker, Docker Compose, Nginx |

6.5 Completeness Assessment

| Documentation Item | Status | Comment |
|----------------------|----------|--|
| Project overview | Complete | Clearly explains purpose and scope. |
| Setup / installation | Complete | Docker-based setup is reproducible. |
| API reference | Partial | Core endpoints documented; a few forecasting endpoints missing examples. |
| Architecture diagram | Present | Included in docs/architecture-diagram.png. |

| | | |
|-------------------------|---------|---|
| Contribution guidelines | Missing | No CONTRIBUTING.md; recommended addition. |
| License | Present | MIT license included at repository root. |

6.6 Suggested Improvements

- Add a CONTRIBUTING.md describing branch naming, commit conventions, and PR review expectations for new collaborators.
- Expand docs/api-reference.md to cover all forecasting-engine endpoints with example payloads.
- Add a troubleshooting section to the README for common Docker/networking issues.
- Include a changelog (CHANGELOG.md) generated from conventional commit messages.

7. Code Review Findings and Recommendations

7.1 Summary of Findings

Overall, the repository demonstrates a well-structured, service-oriented full-stack application that fulfills the objectives of the assigned problem statement: monitoring reservoir levels, forecasting demand and availability, and supporting drought planning through a containerized platform. Code readability and modularity are generally strong, version control practices follow an appropriate branching model with descriptive commit messages, and core documentation (README, setup, API reference) is largely complete. The main gaps identified are in test coverage for the forecasting engine, some validation logic embedded directly in controllers, and a few missing contribution/changelog documents.

7.2 Recommendations

Code Quality

- Extract request validation into reusable middleware to slim down controller functions.
- Move threshold constants (drought risk, low-storage alerts) into a shared configuration module.
- Add JSDoc/type-hint coverage to remaining untyped utility functions.

Repository Management

- Add a CONTRIBUTING.md and pull-request template to standardize collaboration.
- Enable branch-protection rules requiring passing CI checks before merge.

Documentation

- Complete the API reference for all forecasting-engine endpoints.
- Add a CHANGELOG.md and a troubleshooting section to the README.

Testing

- Increase unit test coverage for the forecasting-engine module, particularly edge cases with sparse or missing data.
- Add end-to-end tests covering the full ingestion-to-dashboard data flow.

Future Enhancements

- Incorporate satellite-based rainfall data sources to improve forecast accuracy.
- Add role-based access control for multiple authority levels (district vs. state administrators).
- Introduce model monitoring to detect forecast drift and trigger retraining.

7.3 Conclusion

The AI-Powered Smart Water Reservoir Management and Demand Forecasting Platform, as implemented, meets the core objectives of the assigned problem statement while following sound software engineering practices in repository organization, version control, and documentation. Addressing the recommendations above — particularly around test coverage and contribution documentation — would further strengthen the project's maintainability and readiness for collaborative, long-term development.